

06-差分

06-差分

一、本阶段目标

差分是前缀和的“反向操作”。

这一章的目标是：

- 理解为什么区间修改暴力太慢
- 理解差分数组到底记录了什么
- 掌握一维差分
- 掌握二维差分
- 理解“打标记”的思想
- 掌握最后如何还原原数组
- 理解差分与前缀和的关系

差分的核心思想是：

不要立刻修改整个区间，
只记录“变化从哪里开始、从哪里结束”。

二、为什么需要差分？

1. 普通区间修改

例如：

数组：
1 2 3 4 5

现在要求：

给 [2,4] 全部 +3

普通做法：

```
for(int i=2; i<=4; i++)  
{
```

```
a[i] += 3;  
}
```

复杂度：

$O(n)$

2. 如果修改很多次怎么办？

例如：

有 10^5 次区间修改

如果每次都暴力改：

$O(n \times q)$

会非常慢。

于是我们想到：

能不能只记录“哪里开始变”“哪里结束变”？

这就是差分。

三、差分数组是什么？

1. 定义

定义：

$d[i] = a[i] - a[i-1]$

也就是：

记录相邻元素之间的变化量。

2. 例子

原数组：

```
a:  
1 3 6 10
```

差分数组：

```
d:  
1 2 3 4
```

因为：

```
d[1]=1  
d[2]=3-1=2  
d[3]=6-3=3  
d[4]=10-6=4
```

3. 差分和前缀和的关系

前缀和：

```
通过不断累加得到和
```

差分：

```
通过不断累加恢复原数组
```

也就是说：

```
差分数组的前缀和  
=  
原数组
```

四、一维差分核心思想

1. 区间加法

现在：

给 $[l, r]$ 全部 $+c$

差分操作：

```
d[l] += c;  
d[r+1] -= c;
```

2. 为什么这样就行？

例如：

原数组：

0 0 0 0 0

现在：

给 $[2, 4]$ $+3$

差分：

```
d[2] += 3  
d[5] -= 3
```

差分数组：

0 3 0 0 -3

做前缀和恢复：

```
0  
3  
3  
3  
0
```

也就是：

0 3 3 3 0

成功实现：

[2,4] 全部 +3

五、一维差分真正记录的是什么？

差分记录的是：

“变化从哪里开始”

“变化从哪里结束”

例如：

```
d[l]+=c;
```

表示：

从 l 开始，后面的数都多 c。

而：

```
d[r+1]-=c;
```

表示：

从 r+1 开始，不再多 c。

六、一维差分模板

1. 区间加法模板

```
#include<iostream>
using namespace std;

int main()
{
    int n,m;
    cin>>n>>m;
```

```
int d[100]={0};

while(m--)
{
    int l,r,c;
    cin>>l>>r>>c;

    d[l]+=c;
    d[r+1]-=c;
}

int cur=0;

for(int i=1;i<=n;i++)
{
    cur+=d[i];
    cout<<cur<<" ";
}

return 0;
}
```

2. 为什么最后要恢复？

因为：

差分数组不是最终答案。

它只是：

变化记录表。

最后必须：

做一次前缀和

才能恢复真正数组。

七、在原数组基础上建差分

1. 如果原数组不是全 0

例如：

```
1 2 3 4 5
```

需要先建差分数组。

2. 建差分公式

```
d[i]=a[i]-a[i-1];
```

3. 模板

```
#include<iostream>
using namespace std;

int main()
{
    int n,m;
    cin>>n>>m;

    int a[100];
    int d[100]={0};

    for(int i=1;i<=n;i++)
    {
        cin>>a[i];
    }

    d[1]=a[1];

    for(int i=2;i<=n;i++)
    {
        d[i]=a[i]-a[i-1];
    }

    while(m--)
    {
        int l,r,c;
        cin>>l>>r>>c;

        d[l]+=c;
        d[r+1]-=c;
    }
}
```

```
    }

    for(int i=1;i<=n;i++)
    {
        d[i]+=d[i-1];
    }

    for(int i=1;i<=n;i++)
    {
        cout<<d[i]<<" ";
    }

    return 0;
}
```

八、二维差分

1. 为什么需要二维差分？

如果题目变成：

给矩阵某个矩形区域全部 +c

例如：

给一个矩形区域全部加 1

暴力修改：

会很慢。

于是出现：

二维差分。

九、二维差分核心思想

二维差分本质是：

给矩形区域“打补丁”。

只修改：

四个角

最后统一恢复。

十、二维差分公式

1. 给矩形加 c

矩形：

(x1,y1)
到
(x2,y2)

操作：

```
d[x1][y1] += c;  
d[x2+1][y1] -= c;  
d[x1][y2+1] -= c;  
d[x2+1][y2+1] += c;
```

2. 为什么是四个角？

可以理解成：

从左上角开始 +c

但：

下面不要了
右边不要了
右下角被减重了，所以补回来

和二维前缀和非常像。

十一、二维差分恢复

1. 恢复公式

```
a[i][j]
=
a[i-1][j]
+a[i][j-1]
-a[i-1][j-1]
+d[i][j]
```

其实就是：

二维前缀和。

十二、二维差分模板

```
#include<iostream>
using namespace std;

int main()
{
    int n,m,q;
    cin>>n>>m>>q;

    int d[105][105]={0};

    while(q--)
    {
        int x1,y1,x2,y2,c;
        cin>>x1>>y1>>x2>>y2>>c;

        d[x1][y1]+=c;
        d[x2+1][y1]-=c;
        d[x1][y2+1]-=c;
        d[x2+1][y2+1]+=c;
    }

    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=m;j++)
        {
            d[i][j]
            =
            d[i-1][j]
            +d[i][j-1]
            -d[i-1][j-1]
```

```
        +d[i][j];
    }
}

for(int i=1;i<=n;i++)
{
    for(int j=1;j<=m;j++)
    {
        cout<<d[i][j]<<" ";
    }

    cout<<"\n";
}

return 0;
}
```

十三、二维差分为什么像“打补丁”？

因为：

并没有立刻修改整个区域。

而只是：

记录：
这个矩形从哪里开始生效，
从哪里结束生效。

最后统一恢复。

十四、差分和前缀和的关系

前缀和

作用：

快速查询

特点：

修改慢
查询快

差分

作用：

快速修改

特点：

修改快
查询前需要恢复

两者互逆

前缀和是“累加”
差分是“相邻做减法”

十五、差分真正优化了什么？

差分真正优化的是：

大量区间修改。

普通做法：

每次修改都遍历整个区间

差分：

只改边界
最后统一恢复

十六、常见错误

1. 忘记 r+1

错误：

```
d[l]+=c;  
d[r]-=c;
```

正确：

```
d[l]+=c;  
d[r+1]-=c;
```

2. 恢复时忘记做前缀和

差分数组不是答案。

必须：

```
d[i]+=d[i-1];
```

恢复。

3. 二维差分四个角写错

正确：

```
d[x1][y1]+=c;  
d[x2+1][y1]-=c;  
d[x1][y2+1]-=c;  
d[x2+1][y2+1]+=c;
```

最容易错：

```
右下角符号。
```

4. 数组开小

因为会访问：

```
r+1
x2+1
y2+1
```

所以数组通常要：

```
多开一圈。
```

5. 下标从 0 开始容易乱

差分通常：

```
建议从 1 开始存。
```

十七、本专题做过的题

题号	题目	类型	题面简述	对应知识点
P2367	语文成绩	一维差分	多次区间加法后输出最终成绩	差分区间修改
地毯覆盖类题	二维差分	多个矩形覆盖后统计结果	二维差分	
二维矩阵修改类题	二维差分	多次修改矩形区域	四角标记	
农田灌溉类题	差分	大量区间修改	差分快速更新	

十八、本章最重要的结论

差分的核心不是公式：

```
d[l]+=c;
d[r+1]-=c;
```

真正重要的是：

不要立刻修改整个区间，
只记录“变化从哪里开始、从哪里结束”。

一维差分：

记录区间变化

二维差分：

给矩形打补丁

以后看到：

大量区间修改
大量矩形修改

就应该想到：

差分。